

实验 2 键控机器人运动

一、实验目的

- 1、了解用于描述机器人模型的 urdf 文件，理解 urdf 文件中的 robot、link、joint 标签。
- 2、掌握并熟练运用可视化工具 RViz，包括工具启动、机器人模型加载、TF 可视化等。
- 3、理解 ROS 中树状坐标系的构建逻辑和多坐标系的变换原理，熟练运用 TF 工具。
- 4、了解机器人小车的硬件结构，将仿真模型中的抽象概念（如连杆、关节）与物理小车的实际硬件进行映射对应。
- 5、实验过程中，严格遵守实验室安全操作规范，确保人员与设备安全。

二、实验环境与实验器材

- 1、实验硬件设备
计算机、机器人小车
- 2、实验软件配置
Ubuntu20.04, ros-noetic

三、实验内容

1、仿真实验

要求：加载 urdf 文件构建的小车模型，键盘控制仿真小车运动。

具体流程如下：

第 1 步：安装 teleop_twist_keyboard 功能包

命令如下：

```
$ sudo apt install ros-noetic-teleop-twist-keyboard
```

teleop_twist_keyboard 是 ROS 中一个非常经典且基础的功能包（Package），允许用户通过计算机键盘来控制机器人的运动速度。

第 2 步：载入机器人模型。

1) 创建工作空间并初始化

```
$ mkdir -p ~/catkin_ebot_sim/src  
$ cd ~/catkin_ebot_sim  
$ catkin_make //初始化工作空间
```

2) 创建 ros 功能包

进入 src 文件夹，创建功能包 vehicle_sim，并添加依赖 urdf 和 xacro。

命令如下：

```
$ cd src  
$ catkin_create_pkg vehicle_sim urdf xacro
```

3) 添加 urdf 文件

将 demo02_teleop/sim 文件夹里面的 urdf、mesh 两个文件夹直接复制到 vehicle_sim

文件夹下。

```
$ cp -r /path/to/sim/urdf ~/catkin_ebot_sim/src/vehicle_sim
$ cp -r /path/to/sim/mesh ~/catkin_ebot_sim/src/vehicle_sim
```

注：在 wsl 安装的 ubuntu 镜像系统中，Windows 磁盘挂载在 ubuntu 系统的/mnt 路径下。例如，Windows 的 D 盘在 ubuntu 系统中访问路径为/mnt/d。

4) 编写 launch 文件

```
$ cd ~/catkin_ebot_sim/src/vehicle_sim
$ mkdir launch
$ cd launch
$ gedit vehicle_sim_rviz.launch
```

vehicle_sim_rviz.launch 文件输入内容示例以下：

```
<launch>
  <!-- 在参数服务器载入机器人模型（urdf 文件） -->
  <param name="robot_description" command="$(find xacro)/xacro $(find
vehicle_sim)/urdf/vehicle_top.urdf.xacro" />
  <!-- 启动 rviz -->
  <node pkg="rviz" type="rviz" name="rviz" />
  <!-- 启动机器人关节状态发布节点，发布话题：/joint_states -->
  <node pkg="joint_state_publisher" type="joint_state_publisher"
name="joint_state_publisher" output="screen" />
  <!-- 启动机器人状态发布节点，订阅话题：/joint_states，发布小车 TF 信息-->
  <node pkg="robot_state_publisher" type="robot_state_publisher"
name="robot_state_publisher" output="screen" />
  <!-- 启动图形化的控制关节运动节点 -->
  <node pkg="joint_state_publisher_gui" type="joint_state_publisher_gui"
name="joint_state_publisher_gui" output="screen" />
</launch>
```

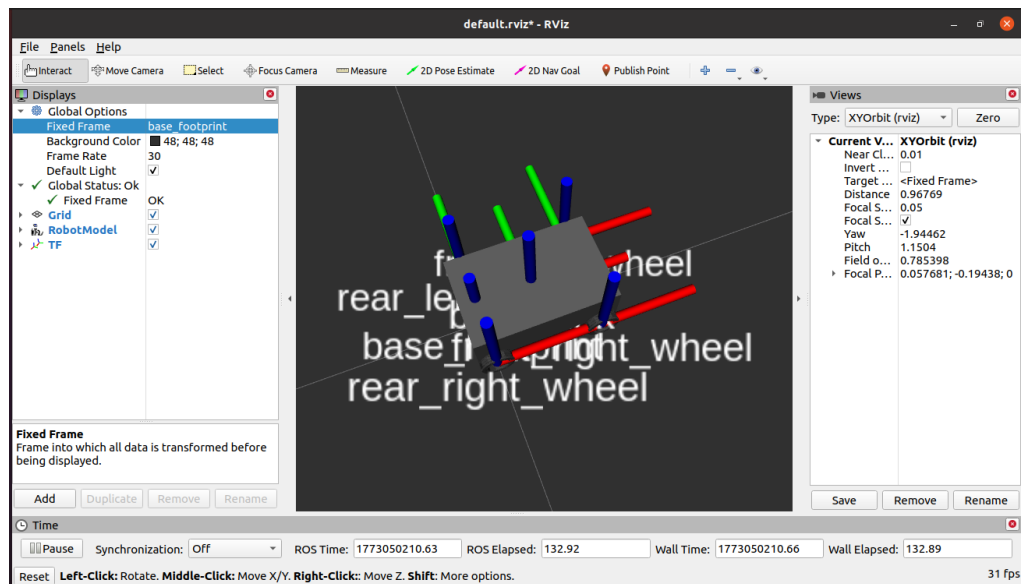
5) 编译执行

终端输入以下命令：

```
$ cd ~/catkin_ebot_sim
$ catkin_make
$ source ./devel/setup.bash
$ roslaunch vehicle_sim vehicle_sim_rviz.launch
```

6) 用 rviz 工具可视化机器人模型

启动 rviz 后，在 Displays 窗口(左侧窗口栏)中，将 Fixed Frame 改为/base_footprint，点击左下按钮“Add”，在 By display type 中寻找 RobotModel 和 TF，依次添加，完成后，即可在中间黑色区域显示小车模型，如下图所示：



注：1）此处 rviz 通过 launch 文件启动，也可在 terminal 终端输入 rviz 启动。

2）rviz 的配置可通过 File→Save Config As，将现有配置保存至 vehicle_sim 功能包下新建的 rviz 文件夹下 (rviz/vehicle_sim.rviz)，并将路径添加至 launch 文件，以便下次直接加载。

```
<!-- 启动 rviz -->
<node pkg="rviz" type="rviz" name="rviz" args="-d $(find
vehicle_sim)/rviz/vehicle_sim.rviz" />
```

第 3 步：配置脚本文件

1) 复制可执行脚本

将 demo02_teleop/sim 下的 scripts 文件夹直接复制到 vehicle_sim 文件夹下，并给.py 文件添加可执行权限。命令如下：

```
$ cp -r /path/to/sim/scripts ~/catkin_ebot_sim/src/vehicle_sim
$ cd ~/catkin_ebot_sim/src/vehicle_sim/scripts
$ chmod +x vehicle_sim.py
```

注：vehicle_sim.py 脚本文件注册 vehicle_sim_node 节点，该节点订阅/cmd_vel 话题，发布/odom 话题。

2) 配置 CMakeLists.txt 和 package.xml

设置可执行脚本并添加依赖：rospy、std_msgs、geometry_msgs、nav_msgs、tf2、tf2_ros、tf2_geometry_msgs。

修改 CMakeLists.txt:

```
find_package(catkin REQUIRED COMPONENTS
  geometry_msgs
  nav_msgs
  rospy
  std_msgs
  tf2
```

```

tf2_ros
tf2_geometry_msgs
urdf
xacro
)
//设置用于安装的可执行脚本
//找到 catkin_install_python 所在位置（约第 168 行），添加
catkin_install_python(PROGRAMS
  scripts/vehicle_sim.py
  DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
)

```

修改 package.xml，在相应位置添加如下内容：

```

<build_depend>geometry_msgs</build_depend>
<build_depend>nav_msgs</build_depend>
<build_depend>rospy</build_depend>
<build_depend>std_msgs</build_depend>
<build_depend>tf2</build_depend>
<build_depend>tf2_ros</build_depend>
<build_depend>tf2_geometry_msgs</build_depend>

<build_export_depend>geometry_msgs</build_export_depend>
<build_export_depend>nav_msgs</build_export_depend>
<build_export_depend>rospy</build_export_depend>
<build_export_depend>std_msgs</build_export_depend>
<build_export_depend>tf2</build_export_depend>
<build_export_depend>tf2_ros</build_export_depend>
<build_export_depend>tf2_geometry_msgs</build_export_depend>

<exec_depend>geometry_msgs</exec_depend>
<exec_depend>nav_msgs</exec_depend>
<exec_depend>rospy</exec_depend>
<exec_depend>std_msgs</exec_depend>
<exec_depend>tf2</exec_depend>
<exec_depend>tf2_ros</exec_depend>
<exec_depend>tf2_geometry_msgs</exec_depend>

```

3) 找见 TODO，完善代码

完善 scripts 文件夹下的 vehicle_sim.py 代码：构建并发布/odom 话题，话题数据为 odom_frame 与 base_frame 之间的坐标变换。

第 4 步：编译并执行

1) 修改 launch 文件

在 vehicle_sim_rviz.launch 文件中加入启动 vehicle_sim_node 节点的命令。

```
<!-- 启动 vehicle_sim_node -->
```

```
<node pkg="vehicle_sim" type="vehicle_sim.py" name="vehicle_sim_node"
output="screen" >
    <!-- <param name="base_frame" value="base_link"/> -->
</node>
```

2) 编译执行

终端 1 输入以下命令，载入仿真机器人模型：

```
$ cd ~/catkin_ebot_sim
$ catkin_make
$ source ./devel/setup.bash
$ roslaunch vehicle_sim vehicle_sim_rviz.launch
```

终端 2 输入以下命令，查看话题/cmd_vel 信息：

```
$ rostopic list
$ rostopic info /cmd_vel
```

终端 3 输入以下命令，启动键盘遥控节点：

```
$ rosrunc teleop_twist_keyboard teleop_twist_keyboard.py
```

rviz 启动后，在 Displays 窗口中，将 Fixed Frame 改为/odom，然后按照按键定义操控机器人模型运动。

2、实车实验

要求：启动机器人小车底盘，键盘远程控制小车运动。

具体流程如下：

第 1 步：检测小车底盘的运动功能

1) 连接显示屏，进入小车系统

卸掉小车保护壳，通过 DP 线和 usb 扩展坞将小车的开发板和显示屏、鼠标、键盘连接。用物体支撑小车底盘中心，使车轮离地悬空，避免小车在运动中掉落桌面。拨动左后轮上方拨码开关，启动小车系统。

2) 查看小车底盘硬件连接

在宿主机命令端如下命令，启动容器并进入：

```
$ xhost +local:root 或 (xhost +)
$ docker start noetic_demo
$ docker exec -it noetic_demo bash
```

在宿主机命令端输入：

```
$ ls -l /dev/serial/by-path/
```

在容器终端输入：

```
$ ls -l /dev/serial/by-path/
```

如果两个终端有如下输出，说明容器和底盘通信正常。

```
nvidia@nvidia-desktop:~$ ls -l /dev/serial/by-path/
total 0
lrwxrwxrwx 1 root root 13 1月 1 1970 platform-3610000.usb-usb-0:2.4:1.0 -> ../../ttyACM0
```

```
nvidia@nvidia-desktop:~$ docker exec -it noetic_demo bash
root@nvidia-desktop:/home/nvidia# ls -l /dev/serial/by-path/
total 0
lrwxrwxrwx 1 root root 13 Jan  1  1970 platform-3610000.usb-usb-0:2.4:1.0 -> ../../ttyACM0
```

3) 安装 teleop_twist_keyboard 功能包

查看容器中是否已安装了 teleop_twist_keyboard，容器终端键入如下命令：

```
$ rospack find teleop_twist_keyboard
```

如果输出功能包的安装路径，说明已经安装。如果没有，容器终端键入如下命令进行安装：

```
$ sudo apt install ros-noetic-teleop-twist-keyboard
```

4) 创建小车底盘驱动和启动功能包

创建工作空间：

```
$ mkdir -p ~/test_demo/catkin_ebot/src
```

将 demo02_teleop/ebot 里面的 ebot_description 和 turn_on_wheeltec_robot 文件夹，复制到 src 路径下，命令如下：

```
$ cd ~/test_demo/catkin_ebot/
$ cp -r /path/to/ebot/ebot_description ./src
$ cp -r /path/to/ebot/turn_on_wheeltec_robot ./src
```

创建功能包 ebot_bringup，不用添加依赖。命令如下：

```
$ cd src
$ catkin_create_pkg ebot_bringup
```

5) 编写 launch 文件

```
$ cd ~/catkin_ebot/src/ebot_bringup
$ mkdir launch
$ cd launch
$ gedit bringup.launch
```

bringup.launch 文件输入内容示例以下：

```
<launch>
  <!-- 载入机器人模型 -->
  <node pkg="tf2_ros" type="static_transform_publisher" name="base_to_link"
args="0 0 0 0 0 base_footprint base_link" />
  <param name = "robot_description"  textfile = "$(find
ebot_description)/urdf/mini_akm_robot.urdf"/>
  <!-- 启动机器人状态和关节状态发布节点
  <node      pkg="joint_state_publisher"      name="joint_state_publisher"
type="joint_state_publisher" output="screen"/>
  <node      pkg="robot_state_publisher"      name="robot_state_publisher"
type="robot_state_publisher" output="screen"/>

  <!-- 启动机器人实车底盘 -->
  <include      file="$(find      turn_on_wheeltec_robot)/launch/
turn_on_wheeltec_robot.launch" /> <!-- from base_control to turn_on-->
```

```
</launch>
```

注：第 4）和 5）步的操作如果在容器内完成，宿主机对其内文件无写入权限。可以在 PC 机上创建功能包并编写 launch 文件，然后直接复制到对应文件夹下。

5) 编译并执行

在第 1 个容器终端内键入如下命令，启动小车底盘：

```
$ cd ~/test_demo/catkin_ebot
$ catkin_make
$ source ./devel/setup.bash
$ roslaunch ebot_bringup bringup.launch
```

在第 2 个容器终端内键入如下命令，查看话题/cmd_vel 信息：

```
$ rostopic list
$ rostopic info /cmd_vel
```

在第 3 个容器终端内键入如下命令，启动键盘遥控节点：

```
$ roslaunch teleop_twist_keyboard teleop_twist_keyboard.py
```

按照按键定义操控实体机器人，观测车轮是否会按预期的操作转动。车轮成功转动后，关闭底盘启动节点和键盘控制节点，退出容器终端。

第 2 步：搭建分布式框架

机器人小车作为主机，PC 机作为从机，搭建 ROS 的分布式通信框架。先将小车和 PC 机连接到同一个局域网，然后修改配置文件。

1) 获取主从机信息

启动小车宿主机终端输入：

```
$ hostname -I
$ hostname
```

假设得到的小车 IP 地址是：192.168.124.10，计算机名是：nvidia-desktop。

启动小车容器终端并输入：

```
$ hostname -I
$ hostname
```

如果得到的 IP 地址和计算机名一致，说明小车的容器和宿主机共享同一个网路环境。否则删除容器，通过 docker run 创建新的容器，且必须添加“--network host”参数。

在 PC 机终端分别输入：

```
$ hostname -I
$ hostname
```

假设得到的 PC 机 IP 地址是：192.168.124.9，计算机名是：viola-HP- ProBook-440-G5。综上所述，假设得到的信息如下：

角色	IP 地址	计算机名
主机 (Master)-小车	192.168.124.10	nvidia-desktop
从机 (Slave)-PC 机	192.168.124.9	viola-HP-ProBook-440-G5

注：默认情况下，WSL2 使用 NAT (网络地址转换) 模式，内部的 Ubuntu 系统拥有一个与宿主机（Windows）完全不同的 IP 网段，使得 Ubuntu 系统无法直接与局域网内的其他设备进行通信，需将 WSL2 的网络模式从 NAT 模式改为镜像模式 (Mirrored Mode)。具体操作如下：

首先，关闭 WSL，命令为：\$ wsl --shutdown。

其次，在 Windows 系统中，进入 C:\Users\<你的用户名>\目录，找到或新建一个名为 .wslconfig 的文件，打开并输入以下内容：

```
[wsl2]
networkingMode=mirrored
[experimental]
hostAddressLoopback=true
```

保存并退出后，重启 WSL，分别查看宿主机（Windows）和 WSL2 内部的 Ubuntu 系统的 ip 地址，命令分别为：\$ ipconfig 和 \$ hostname -I。

2) 配置 Hosts 文件

修改/etc/hosts 文件，分别添加对方的信息，让两台机器能通过名字找到对方。

修改主机（小车）的 hosts 文件，在小车宿主机上打开终端，编辑 hosts 文件：

```
$ sudo gedit /etc/hosts
```

在文件末尾添加从机的 IP 和名字，如下图所示：

```
1 127.0.0.1      localhost
2 127.0.1.1      nvidia-desktop
3 192.168.124.9  viola-HP-ProBook-440-G5
```

修改从机（PC 机）的 hosts 文件，在 PC 机上打开终端，编辑 hosts 文件：

```
$ sudo gedit /etc/hosts
```

在文件末尾添加主机的 IP 和名字，如下图所示：

```
1 127.0.0.1      localhost
2 127.0.1.1      viola-HP-ProBook-440-G5
3 192.168.124.10 nvidia-desktop
```

3) 配置环境变量

配置主机环境变量，在小车宿主机上启动容器终端，打开.bashrc 文件：

```
$ gedit ~/.bashrc
```

弹出.bashrc 文件，在文件的最后加入如下三行命令：

```
$ export ROS_MASTER_URI=http://192.168.124.10:11311
$ export ROS_IP=192.168.124.10
$ export ROS_HOSTNAME=192.168.124.10
```

保存后关闭.bashrc 文件，然后在终端输入如下命令：

```
$ source ~/.bashrc
```

配置从机环境变量，在 PC 机上打开终端，编辑.bashrc 文件

```
$ sudo gedit ~/.bashrc
```


输入密码后会弹出.bashrc 文件，在文件的最后加入如下两行命令：

```
export ROS_MASTER_URI=http://192.168.124.10:11311
export ROS_IP=192.168.124.9
```

保存后关闭.bashrc 文件，然后在终端输入如下命令：

```
$ source ~/.bashrc
```

4) 检测主从机通信功能

在小车宿主机终端输入如下命令：

```
$ ping 192.168.124.9
```

在 PC 机终端输入如下命令：

```
$ ping 192.168.124.10
```

如果输出如下的结果，说明 PC 机与小车之间的网络双向连通成功。

```
nvidia@nvidia-desktop:~$ ping 192.168.124.9
PING 192.168.124.9 (192.168.124.9) 56(84) bytes of data.
 64 bytes from 192.168.124.9: icmp_seq=1 ttl=64 time=1196 ms
 64 bytes from 192.168.124.9: icmp_seq=2 ttl=64 time=168 ms
 64 bytes from 192.168.124.9: icmp_seq=3 ttl=64 time=88.3 ms
 64 bytes from 192.168.124.9: icmp_seq=4 ttl=64 time=212 ms
^C
--- 192.168.124.9 ping statistics ---
 4 packets transmitted, 4 received, 0% packet loss, time 3032ms
 rtt min/avg/max/mdev = 88.293/416.043/1196.429/452.726 ms, pipe 2
```

```
viola@viola-HP-ProBook-440-G5:~$ ping 192.168.124.10
PING 192.168.124.10 (192.168.124.10) 56(84) bytes of data.
 64 字节，来自 192.168.124.10: icmp_seq=1 ttl=64 时间=2621 毫秒
 64 字节，来自 192.168.124.10: icmp_seq=2 ttl=64 时间=1618 毫秒
 64 字节，来自 192.168.124.10: icmp_seq=3 ttl=64 时间=594 毫秒
 64 字节，来自 192.168.124.10: icmp_seq=4 ttl=64 时间=2.78 毫秒
 64 字节，来自 192.168.124.10: icmp_seq=5 ttl=64 时间=3.54 毫秒
 64 字节，来自 192.168.124.10: icmp_seq=6 ttl=64 时间=47.5 毫秒
 64 字节，来自 192.168.124.10: icmp_seq=7 ttl=64 时间=4.82 毫秒
 64 字节，来自 192.168.124.10: icmp_seq=8 ttl=64 时间=401 毫秒
 64 字节，来自 192.168.124.10: icmp_seq=9 ttl=64 时间=3.26 毫秒
^C
--- 192.168.124.10 ping 统计 ---
已发送 9 个包，已接收 9 个包，0% 包丢失，耗时 8037 毫秒
 rtt min/avg/max/mdev = 2.775/588.352/2621.190/874.801 ms, 管道 3
```

在小车容器终端内启动小车底盘，命令如下：

```
$ cd ~/test_demo/catkin_ebot
$ source ./devel/setup.bash
$ roslaunch ebot_bringup bringup.launch
```

在 PC 机终端输入如下命令：

```
$ rostopic list
$ rostopic echo /tf_static
```

如果输出右图的结果，说明主机启动发布节点，从机启动订阅节点后，通信正常。

```

viola@viola-HP-ProBook-440-G5:~$ rostopic list
/Chassis
/PowerVoltage
/cmd_vel
/imu
/joint_states
/rosout
/rosout_agg
/simple_udp/simple_int8
/tf
/tf_static
/ultrasonic_sensors/out/front_left

viola@viola-HP-ProBook-440-G5:~$ rostopic echo /tf_static
transforms:
-
  header:
    seq: 0
    stamp:
      secs: 1774430213
      nsecs: 852196795
    frame_id: "base_footprint"
  child_frame_id: "base_link"
  transform:
    translation:
      x: 0.0
      y: 0.0
      z: 0.0

```

在 PC 机终端输入如下命令：

```
$ rosrun teleop_twist_keyboard teleop_twist_keyboard.py
```

按照按键定义操控实体机器人，观测车轮是否会按预期的操作转动。车轮成功转动后，说明主机启动订阅节点，从机启动发布节点后，通信正常。测试完成后，退出小车和 PC 机的所有终端。

注：1) roscore 必须在主机端启动。

2) 在测试过程中，必须对正向和反向通信都进行测试，即主机启动发布节点，从机启动订阅节点，测试通信是否正常；主机启动订阅节点，从机启动发布节点，测试通信是否正常。

第 3 步：远程控制小车运动

1) 开启小车 SSH 服务

在小车宿主机终端查看 SSH 服务端状态，命令如下：

```
$ sudo systemctl status ssh
```

如果看到类似 `ssh.service - OpenBSD Secure Shell server` 信息，说明系统中已安装 SSH 服务端，继续查看运行状态，如果是 `active (running)`，说明 SSH 服务已开启，且正在运行，可以进行远程访问。

如果运行状态是 `inactive (dead)`，说明 SSH 服务端虽已安装但未启动，需开启 SSH 服务，命令如下：

```
$ sudo systemctl enable --now ssh
```

如果看到报错信息 `Unit ssh.service could not be found`，说明系统中没有安装 SSH 服务端，先安装 `openssh-server`，命令如下：

```
$ sudo systemctl status ssh
```

安装完成后，SSH 服务默认会自动启动，且会被设置为开机自启。

2) 通过 SSH 远程连接机器人小车

方式一：直接使用命令行

在 PC 机终端输入命令：`ssh 用户名@服务器地址`。

首次连接时，系统会提示确认服务器的指纹，输入 `yes` 后按回车，然后根据提示输入对应用户的密码即可完成登录。

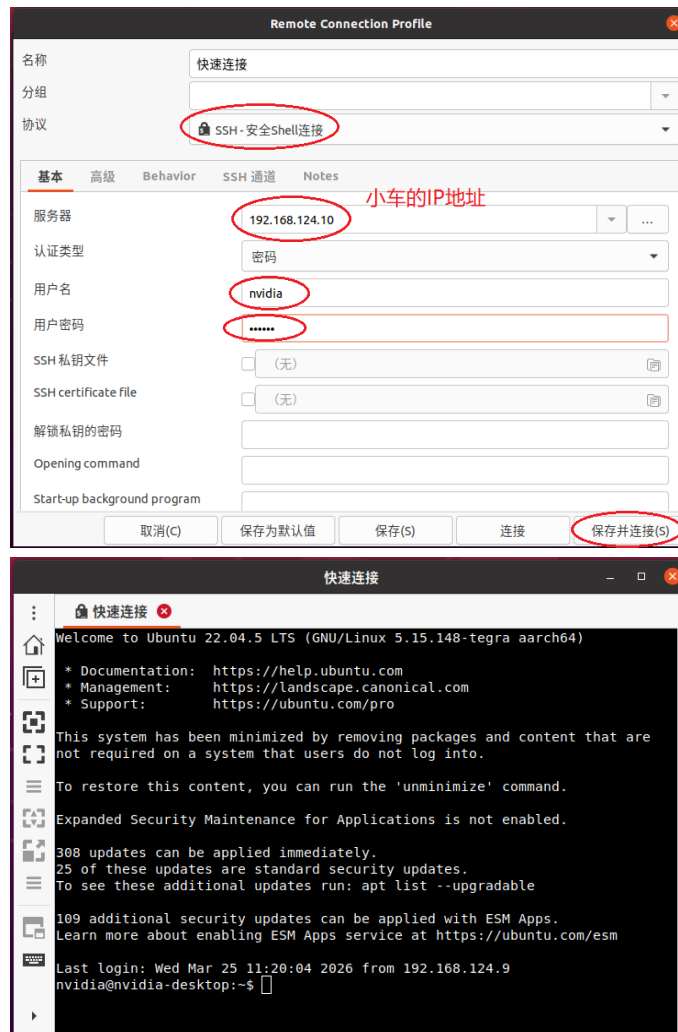
注：使用 `ssh` 命令登录远程服务器时，使用用户名（不是计算机名或主机名），即命令行提示符 `@` 符号前面的部分，查看命令为 `whoami`。

方式二：使用图形化的客户端软件 remmina

在 PC 机终端输入命令\$ remmina，启动 remmina 远程桌面客户端。如果终端提示 Command 'remmina' not found，说明 remmina 未安装，安装命令如下：

```
$ sudo apt install remmina remmina-plugin-rdp remmina-plugin-vnc remmina-plugin-secret
```

点击左上角的“+”号新建连接，协议选择 SSH 连接，输入小车的 ip 地址、用户名和密码，点击保存并连接，进入远程连接终端。



2) 在远程连接终端启动容器

命令如下：

```
$ xhost +local:root 或 (xhost +)  
$ docker start noetic_demo  
$ docker exec -it noetic_demo bash
```

3) 启动小车底盘

在容器终端内键入如下命令：

```
$ cd ~/test_demo/catkin_ebot  
$ source ./devel/setup.bash
```

```
$ roslaunch ebot_bringup bringup.launch
```

4) 启动键盘遥控功能包

在容器终端或 PC 机终端键入如下命令：

```
$ rosrn teleop_twist_keyboard teleop_twist_keyboard.py
```

按提示操控实体机器人，观测车轮是否会按预期的操作运动。车轮成功转动后，即可拔掉机器人小车上的 DP 线和和 usb 扩展坞，将小车放置到地面上，在 PC 机上按提示操控机器人运动。

四、实验要求

1、配置 ros 环境

部署机器人小车的 ros 环境。

参阅资料：小车 docker 安装 ros-noetic.zip

2、仿真实验

1) 完善 scripts 文件夹下的 vehicle_sim.py 代码：构建并发布 odom_frame 与 base_frame 之间的坐标变换。

2) 用 rqt_tf_tree 或 tf2_tools 查看 TF（坐标变换）的树结构。

3、实车实验

1) 启动小车底盘，并远程控制小车运动。

2) 用 rqt_tf_tree 或 tf2_tools 查看 TF（坐标变换）的树结构。

4、思考问题

1) 给出/odom 坐标系与/odom 话题的含义，并分析两者的区别。

2) 给出/cmd_vel 和/odom 话题的发布方和订阅方，分析两者之间的关系。

五、实验报告要求

1. 提交时间：两周之内提交

2. 页数：2~4 页以内

3. 报告内容：实验目的、实验步骤与结果分析、结论与问题讨论

4. 提交方式：电子版和纸质版均需提交；

电子版：实验报告和代码打包，加密压缩后，上传至网盘 <http://58.206.101.71:8266/>；

压缩包名：班级+学号+姓名，报告格式为 word 或 pdf；

纸质版：附上电子版密码，黑白双面打印即可，由各班学委收齐后交至科学馆 303。

六、参考资料

1. 张新钰，赵虚左，丘楠，郭世纯，《ROS 机器人理论与实践》，清华大学出版社

注：本次实验主要涉及《ROS 机器人理论与实践》第五章和第六章内容，重点参阅 5.1、6.1-6.5 内容。